SIMILARITY-BASED DUPLICATE RECORD DETECTION SYSTEM

A Project Report Submitted in

the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Allipuram Akhil – 2420030540

D. Surya – 2420030440

P. Ruthvik Reddy – 2420030546

Under the Esteemed Guidance of

Ms. Bonagani Prathusha
Assistant Professor

Department of Computer Science and Engineering

AY-2026–2027

Semester: Odd

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**Similarity-Based Duplicate Record Detection Using String Matching and Machine Learning**" is a record of Bonafide work of **Allipuram Akhil – 2420030540, D. Surya – 2420030440, P. Ruthvik Reddy – 2420030546** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Allipuram Akhil – 2420030540

D. Surya – 2420030440

P. Ruthvik Reddy – 2420030546

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report entitled "**Similarity-Based Duplicate Record Detection Using String Matching and Machine Learning**" is a bonafide work done and submitted by **Allipuram Akhil – 2420030540, D. Surya – 2420030440, P. Ruthvik Reddy – 2420030546** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

_____	_____	_____
Signature of the Guide	Course Coordinator	Head of the Department
Ms. Bonagani Prathusha		Dr. P. Venkateswara Rao
Assistant Professor, CSE	Department of CSE	Department of CSE

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We would like to express our sincere thanks to everyone who supported us, in one way or another, in completing this work.

We are especially grateful to our project guide, **Ms. Bonagani Prathusha, Assistant Professor,** Department of Computer Science and Engineering, whose steady support, encouragement and guidance cannot be put into words and shaped every stage of this project.

We express our gratitude to **Dr. P. Venkateswara Rao**, Head of the **Department of Computer Science and Engineering**, for providing us with the facilities and the academic environment needed to complete this project.

We thank all the teaching and non-teaching staff who assisted us directly or indirectly in completing this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Allipuram Akhil – 2420030540

D. Surya – 2420030440

P. Ruthvik Reddy – 2420030546

ABSTRACT

Duplicate customer records are a quiet but costly problem. A customer's history ends up spread across several rows, counts come out too high, and any analysis built on the data becomes less reliable. Because duplicates rarely match exactly, a plain equality check cannot find them.

This project, "Similarity-Based Duplicate Record Detection Using String Matching and Machine Learning", builds and tests a system that covers the full workflow: text normalisation, blocking, multi-field similarity scoring, supervised classification, clustering of duplicates and an interactive dashboard.

Every candidate pair of records is described by Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine and exact-match features. Logistic Regression, Random Forest and XGBoost are then compared on exactly the same feature set. The test data is a generated set of 503 customer records, made of 350 originals and 153 injected duplicates.

Blocking reduced the 126,253 possible pairs to 605 candidates. Of these, 172 were flagged as duplicates, which left 357 representative records, a reduction of 146 rows (29.0%). On 182 evaluated pairs all three classifiers reported an accuracy of 0.995, precision of 1.000, recall of 0.981 and F1-score of 0.990.

Because the data is synthetic and the train–test split is only partly documented, these figures are treated as a promising demonstration and not as evidence of real-world performance. The report lists the validation steps that should follow. The system is implemented in Python with scikit-learn, XGBoost, Flask and Streamlit.

Keywords: Duplicate Detection, Entity Resolution, Record Linkage, String Similarity, Blocking, Levenshtein Distance, Jaro–Winkler, TF–IDF, Machine Learning, Data Quality.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	4
		Abstract	5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users/Audience 1.7 Related Work 1.8 Research Gap and Problem Formulation	1
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	8
3	Technology Stack	3.1 Front-end – Flask and Streamlit 3.2 Application Logic – Python, pandas and NumPy 3.3 Machine Learning – scikit-learn and XGBoost 3.4 String Similarity – Levenshtein and jellyfish 3.5 Data Visualization – Matplotlib, Seaborn and Plotly 3.6 Output Storage – CSV and JSON	11
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of System Modules 4.3 Application Execution Architecture	13
5	Design	5.1 User Interface Design and Screenshots 5.2 System Workflow / Data Flow Diagram 5.3 Dataset Design 5.4 Data Preprocessing Design	15
6	Implementation	6.1 Module-wise Implementation 6.2 Data Ingestion and Validation 6.3 Candidate Generation (Blocking) 6.4 Similarity Feature Engineering 6.5 Classification Models 6.6 Record-Level Consolidation 6.7 Prediction and Performance Analysis	20
7	Features	7.1 List of Main Features 7.2 Working of Each Feature 7.3 Interactive Dashboard 7.4 Output Artifacts	27
8	Testing	8.1 Functional Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results 8.5 Model Performance Evaluation 8.6 Threats to Validity	30
9	Deployment	9.1 Steps to Run the Application 9.2 Python Environment Configuration	35

		9.3 Local Application Execution 9.4 Production Readiness	
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations	37
11	Future Enhancements	11.1 Planned Features 11.2 Learned Weights and Transformer Matching 11.3 Human Review and Scalability	40
12	Conclusion	12.1 Summary of the Project 12.2 Achievements 12.3 Skills Learned During Development	42
13	References	Research Papers, Books and Technical Documentation	44
14	Appendices	Application Screenshots Sample Code Snippets Installation and Setup Instructions User Manual Team Details	46

CHAPTER 1

Introduction

1.1 Background of the Project

Customer and entity data rarely comes from one clean source. It is typed into websites, collected by mobile apps and point-of-sale systems, logged at service desks and carried over from older databases. When the same person or organisation is entered twice, the two copies seldom match exactly. A name may be misspelled, a phone number may pick up dashes, an address may be abbreviated or reordered, or a field may simply be left empty.

A plain equality check cannot tell that such records belong together, so the duplicates stay. They split customer histories across several rows, distort counts, add to storage and maintenance costs and weaken any analysis done on top of the data.

Finding these duplicates is known as record linkage or entity resolution: deciding whether two records describe the same real-world entity. It is hard because no single attribute can be trusted by itself. A dependable system has to look at several fields and put the evidence together, which is roughly what a careful human reviewer would do.

In this project we built a similarity-based duplicate record detection system that follows exactly this idea. It combines data preprocessing, candidate generation through blocking, multi-field similarity computation, supervised classification, duplicate clustering and a web-based analytics interface called “Duplicate Detection Intelligence Studio”.

1.2 Problem Statement

Duplicate records are hard to find because real data is messy. Exact-match rules fail on near duplicates caused by typos, punctuation, capitalisation or

abbreviations. A single similarity function ignores the evidence held in the other fields. Comparing every pair of records becomes very expensive as the data grows, and a bare yes/no prediction is not enough in real use, because people want to see both records, understand why they were matched and review uncertain cases before anything is merged.

Many systems also report classification metrics but never turn the predicted pairs into entity groups or a cleaned dataset. The problem addressed here is therefore to design an end-to-end workflow that goes from raw input, to reviewable pairs, to a downloadable cleaned result, while keeping the count of duplicate pairs separate from the count of rows removed.

Formally, let the input be R = {r1, r2, …, rn}, where every record carries attributes such as name, email, phone and address. The aim is to find the set of pairs M = {(ri, rj)} that refer to the same real entity. For every candidate pair a feature vector is built from field-level comparison functions, and a classifier labels the pair as duplicate or not.

1.3 Scope of the Project

The scope of this project includes:

1. Detecting duplicate records in structured customer data (name, email, phone and address).
2. Normalising text and phone values and flagging rows with missing fields.
3. Generating candidate pairs through blocking, with a neighbouring-name safety net.
4. Computing character-level, token-level, vector-level and exact-match similarity features.
5. Training and comparing Logistic Regression, Random Forest and XGBoost on one shared feature set.
6. Grouping predicted duplicate pairs into clusters using union-find and choosing one representative record per cluster.

7. Producing a pair report, a cleaned unique dataset and a metrics file for download.
8. Providing a web interface for upload, inspection and download of results.

The scope is structured customer-record deduplication. Because the evaluation uses a generated dataset, the results should not be taken as proof of performance on every real customer database.

1.4 Objectives

The main objectives of this project, and the modules that deliver them, are listed in Table 1.1.

Table 1.1: Project objectives and the modules that realise them

No.	Objective	Module
O1	Normalise attributes and find incomplete rows	Preprocessing
O2	Generate candidate pairs (key + neighbouring names)	Blocking
O3	Compute Levenshtein, Jaro–Winkler, Jaccard, TF–IDF and exact features	Similarity engine
O4	Train and compare Logistic Regression, Random Forest and XGBoost	Model layer
O5	Tune by grid search with 5-fold cross-validation (as designed)	Model layer
O6	Produce a pair report and a cleaned dataset	Consolidation
O7	Web interface for upload, inspection and download	Dashboard

In addition, the work makes five main contributions: (1) a modular pipeline that avoids unnecessary comparisons through blocking; (2) a multi-field similarity representation for structured customer records; (3) a side-by-side comparison of three supervised classifiers on one shared feature set; (4) a step that converts pairwise predictions into record clusters and a cleaned dataset; and (5) an interactive interface for inspecting matches and downloading results.

1.5 Importance of the Application

Clean data is the base of every report, model and decision that is built on it. When duplicates are left in a customer table, counts are inflated, customer histories are split and marketing or service actions are repeated.

The application shows how approximate string matching and machine learning can be combined into a usable data-quality workflow. Because the pair scores, the duplicate report and the cleaned dataset are kept as separate outputs, a reviewer can tell model evaluation apart from the final consolidation result, which makes the whole process traceable.

The project also gives practical experience in preprocessing, feature engineering, classifier comparison, clustering and result presentation.

1.6 Target Users/Audience

The intended users of the application include:

1. Data analysts and data engineers who need to clean customer or entity tables.
2. Students learning entity resolution, record linkage and applied machine learning.
3. Faculty members and researchers working on data quality.
4. Developers who want a small, inspectable baseline for duplicate detection.
5. Business teams that maintain CRM, billing or service databases.

1.7 Related Work

Record linkage has a long history in statistics and databases. Fellegi and Sunter proposed a probabilistic framework in which the pattern of agreement and disagreement across attributes decides whether two records are linked [1]. The lesson that has lasted is that evidence from several fields should be weighed together, and no single attribute should decide on its own.

Approximate string comparison is the next building block. Levenshtein distance counts the insertions, deletions and substitutions needed to turn one string into another [2]. Jaro similarity was designed for record linkage and works well on short strings with transposed characters [3], and Winkler's prefix adjustment gives extra credit to names that start alike [4]. A comparison of string distance metrics for name matching found that the best choice depends on the data [14]. Token-based measures give a different view: Jaccard similarity compares token sets and so tolerates reordering, while TF–IDF weights terms by how distinctive they are and compares the resulting vectors using cosine similarity [5], [15]. Christen gives a broad treatment of the whole linkage pipeline [6].

Scalability depends on how candidates are generated. Comparing every pair among n records requires

$$n(n-1)/2 \quad (1)$$

comparisons, a number that grows quadratically. Blocking and indexing shrink the search space by grouping records that share selected attributes [7], [8]. The price is that blocks that are too strict can throw away true duplicates before the classifier ever sees them.

Learning-based approaches treat each candidate pair as a feature vector. Bilenko and Mooney studied learnable string similarity for duplicate detection [9]. Logistic Regression is an interpretable baseline, Random Forest can capture non-linear interactions [10], and XGBoost gives efficient gradient-boosted trees for tabular data [11]. Deep and pretrained-language-model matchers go further still [12], [13].

We chose classical similarity functions and supervised classifiers over a large neural matcher on purpose. This keeps the system easy to inspect and realistic for a student project, and it leaves room for transformer-based matching, active learning and human review later. Table 1.2 shows how each strand of prior work is used in this project.

Table 1.2: Prior work and its role in this project

Topic	Ref.	Contribution used here
Probabilistic linkage	[1]	Combine evidence from many fields
Edit distance	[2]	Name, email and address Levenshtein
Jaro and Jaro–Winkler	[3], [4]	Short-string, prefix-aware name match
String metric comparison	[14]	Justifies using several metrics
TF–IDF and set similarity	[5], [15]	Token overlap and weighted cosine
Linkage pipeline design	[6]	End-to-end workflow design
Indexing and blocking	[7], [8]	Blocking key + neighbour pairs
Learnable similarity	[9]	Supervised pair classification
Random Forest, XGBoost	[10], [11]	Non-linear classifiers compared
Deep matching	[12], [13]	Listed as a future extension

1.8 Research Gap and Problem Formulation

Looking at the literature and at how deduplication is done in practice, a few gaps keep showing up. Table 1.3 links each gap to the design response taken in this work.

Table 1.3: Research gaps and design responses

Gap	Response in this work
Exact match misses near duplicates	Normalisation + approximate similarity
One metric sees one kind of error	Character, token, vector and exact features
All-pairs cost grows quadratically	Blocking + neighbouring-name safety net
Predictions are hard to trust	Searchable pair table with scores
Metrics without cleaning	Union-find clusters + cleaned CSV
Pairs vs. rows confused	Both counts reported separately

There is another gap between model evaluation and data cleaning. A useful workflow should go from raw input to reviewable pairs to a downloadable result. It should also keep the count of pairs separate from the count of rows removed, since one record can take part in several duplicate relationships. In practice the pipeline needs four things: normalised data, a way to generate

candidates, evidence combined across fields, and a final step that consolidates records.

CHAPTER 2

System Requirements

2.1 Hardware Requirements

The system is built to run locally for experiments on a modest machine. The following hardware configuration is suitable for running the project with the 503-record dataset.

```
Processor       : Intel Core i3 / i5 or equivalent
RAM             : 4 GB minimum, 8 GB recommended
Storage         : At least 1 GB of free space
Operating Input : Keyboard and Mouse
Display         : Standard Laptop or Desktop Display
GPU             : Not required
```

A GPU is not needed because the classifiers work on small tabular feature vectors and no neural network is trained.

2.2 Software Requirements

The software components used for the project are listed in Table 2.1.

Table 2.1: Software stack

Component	Role
Python	Core language for processing and machine learning
pandas, NumPy	Tabular data manipulation
scikit-learn	Preprocessing, TF–IDF, model utilities
XGBoost	Gradient-boosting classifier
Python-Levenshtein, jellyfish	Edit-distance and Jaro–Winkler similarity
matplotlib, seaborn	Visual analysis
Flask	Web interface, asynchronous runs and progress updates
Streamlit, Plotly	Dashboard add-on (upload, threshold, charts)

In addition, the operating system can be Windows 10/11 or any platform that supports Python 3.x, and a code editor such as Visual Studio Code or a Jupyter Notebook is used for development.

2.3 Development Tools and Frameworks

Python:

Python is the main language of the project. It is used for text processing, feature computation, model training, clustering and the web applications.

pandas and NumPy:

pandas holds the customer records and the pair table, while NumPy handles numerical arrays and calculations on the feature vectors.

scikit-learn:

scikit-learn provides preprocessing, TF–IDF vectorisation, Logistic Regression, Random Forest, grid search and evaluation metrics.

XGBoost:

XGBoost provides the gradient-boosted tree classifier that is compared against the other two models.

Python-Levenshtein and jellyfish:

These libraries compute edit distance and Jaro–Winkler similarity between strings.

Flask:

Flask serves the main web interface. It can generate a synthetic dataset or accept a CSV file and runs the pipeline asynchronously with progress updates so that the browser stays responsive.

Streamlit and Plotly:

Streamlit and Plotly make up the dashboard add-on with dataset selection or upload, a similarity-threshold control, pair inspection, model comparison, a confusion matrix and downloads.

matplotlib and seaborn:

These libraries are used to draw the score histogram, model comparison and dataset reduction charts.

CHAPTER 3

Technology Stack

3.1 Front-end – Flask and Streamlit

Two interfaces sit on top of the same pipeline. The Flask interface can generate a synthetic customer dataset or accept a CSV of names, emails, phones and addresses. It runs the work asynchronously and reports progress, so the browser does not freeze while features are being computed.

The Streamlit and Plotly add-on is the analytics dashboard, titled "Duplicate Detection Intelligence Studio". It offers dataset selection or upload, a similarity-threshold control for unlabelled data, pair inspection, model comparison, confusion-matrix display and file downloads. The dashboard is meant to make the model's decisions visible instead of hiding them.

3.2 Application Logic – Python, pandas and NumPy

Python holds the application logic. A data layer loads and validates the CSV and creates the normalised fields, a candidate layer applies blocking, a similarity layer computes the string and vector features, a model layer trains the classifiers and predicts pair labels, and a consolidation layer forms clusters and exports the representatives. pandas and NumPy carry the data between these layers.

3.3 Machine Learning – scikit-learn and XGBoost

Three supervised classifiers are evaluated on the same pair representation, so any difference in results comes from the learners and not from the evidence they are given.

Logistic Regression: an interpretable baseline with a linear decision boundary.

Random Forest: an average of many decision trees trained on bootstrapped samples and random feature subsets. It can learn, for example, that a high name similarity counts for more when the phone number also matches exactly.

XGBoost: gradient-boosted trees built one after another, each correcting the errors of the ensemble so far, with regularisation to keep complexity under control.

scikit-learn also supplies TF–IDF vectorisation and cosine similarity, which are used for the weighted text-overlap feature.

3.4 String Similarity – Levenshtein and jellyfish

Python-Levenshtein computes the edit distance used for the normalised Levenshtein similarity, and jellyfish provides Jaro–Winkler similarity. Together with Jaccard token overlap, these give character-level and token-level views of how close two strings are.

3.5 Data Visualization – Matplotlib, Seaborn and Plotly

matplotlib and seaborn are used for static visual analysis, while Plotly drives the interactive charts in the dashboard: the combined-score histogram, the model comparison chart and the dataset reduction chart.

3.6 Output Storage – CSV and JSON

Results are written as plain files so that they can be inspected and reused: a CSV of all candidate pairs with their scores, a readable duplicate report, a cleaned unique dataset and a JSON file of classifier metrics. Keeping them separate helps traceability.

CHAPTER 4

System Architecture

4.1 High-Level Architecture Diagram

The system is a chain of loosely coupled stages (Fig. 4.1). The input layer accepts a CSV file or generates a synthetic customer dataset. Preprocessing standardises text and phone values. Candidate generation groups records that are likely to be related and forms pairs for detailed comparison. The feature layer computes similarity values, the model layer predicts whether a pair is a duplicate, and the consolidation layer groups linked records and picks one representative. The presentation layer then shows the results and offers downloads.

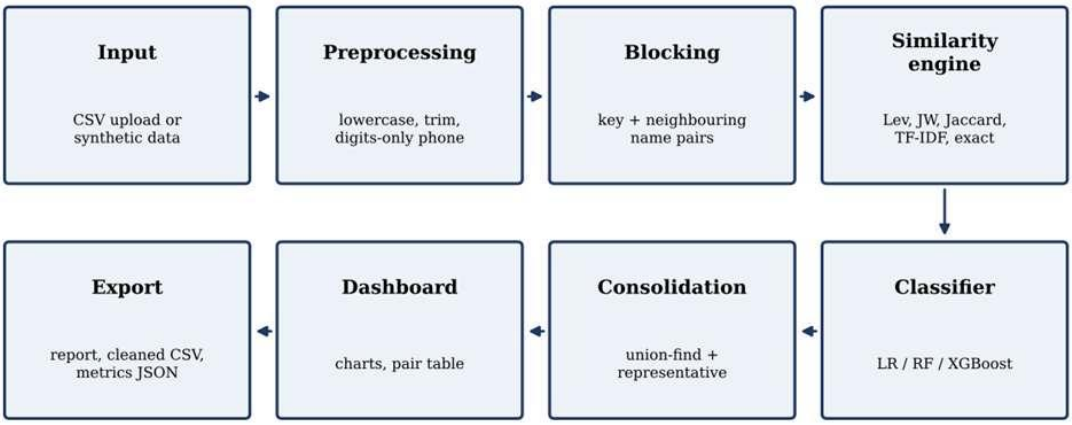


Fig. 4.1: Functional architecture of the similarity-based duplicate detection pipeline

4.2 Description of System Modules

We deliberately keep pair-level matching separate from record-level cleaning. Pair-level matching asks whether two records probably describe the same entity. Consolidation then joins the predicted links into connected components. This separation matters because one entity can appear in three or more rows and

produce several pairwise links, so the number of flagged pairs is not the number of rows removed. Table 4.1 lists the functional modules.

Table 4.1: Functional modules

Module	Responsibility
Dataset ingestion	Read records, validate required columns
Data preparation	Handle missing values, standardise strings and phones
Blocking	Build candidate groups from name and phone
Similarity engine	Character, token, vector and exact-match features
Model evaluation	Train and evaluate the classifiers
Duplicate clustering	Merge links, choose representatives
Dashboard and export	Charts, metrics, pair details, CSV output

4.3 Application Execution Architecture

The system is designed to run locally. The user starts the Flask application or the Streamlit dashboard from the project directory, and a local web interface becomes available in the browser. Uploaded data passes through validation, the pipeline runs, and results are shown and written to files.

A production version would need authentication, audit logging, access restrictions, secure storage and a review workflow for uncertain matches, as discussed in Chapter 9.

CHAPTER 5

Design

5.1 User Interface Design and Screenshots

The dashboard is designed to be simple and to show what the system is doing. Its main components are listed in Table 5.1, and the screenshots from the documented run follow.

Table 5.1: Dashboard components

Component	Purpose
Summary indicators	Input records, candidate pairs, flagged pairs, unique records
Score histogram	Flagged vs. non-flagged similarity
Reduction chart	Input vs. unique records
Model table + chart	Compare three classifiers
Confusion matrix	Error counts of the best model
Searchable pair table	Inspect names, emails, phones, addresses and scores
Threshold slider	Unlabelled data only

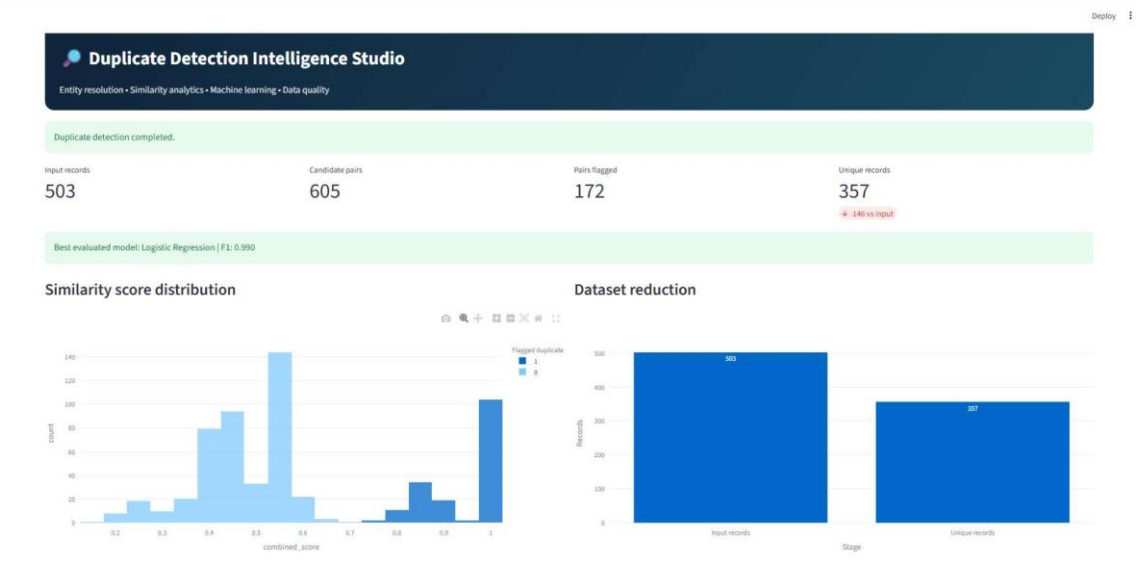


Fig. 5.1: Duplicate Detection Intelligence Studio dashboard overview



Fig. 5.2: Model evaluation comparison for three classifiers

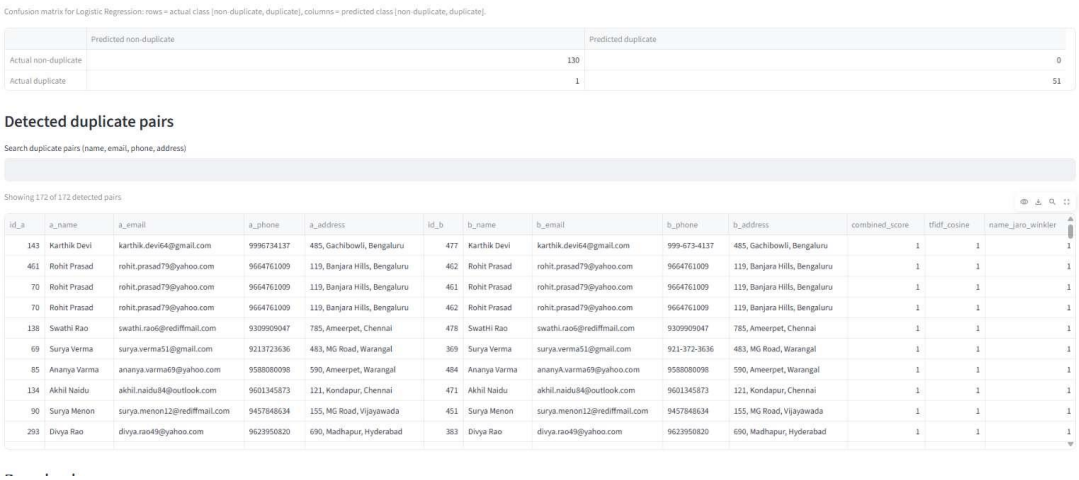


Fig. 5.3: Confusion matrix and detected duplicate-pair table

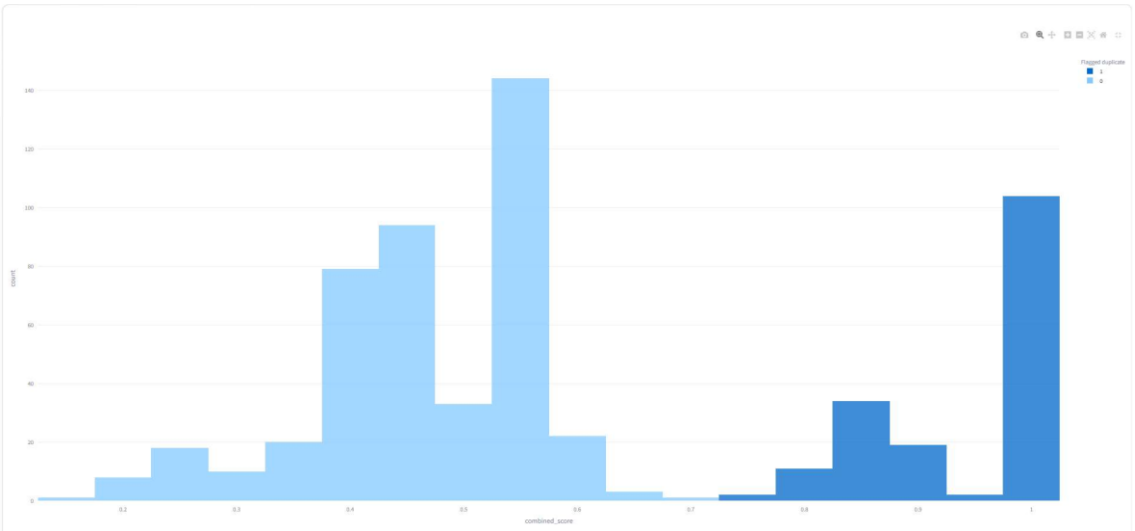


Fig. 5.4: Distribution of combined similarity scores

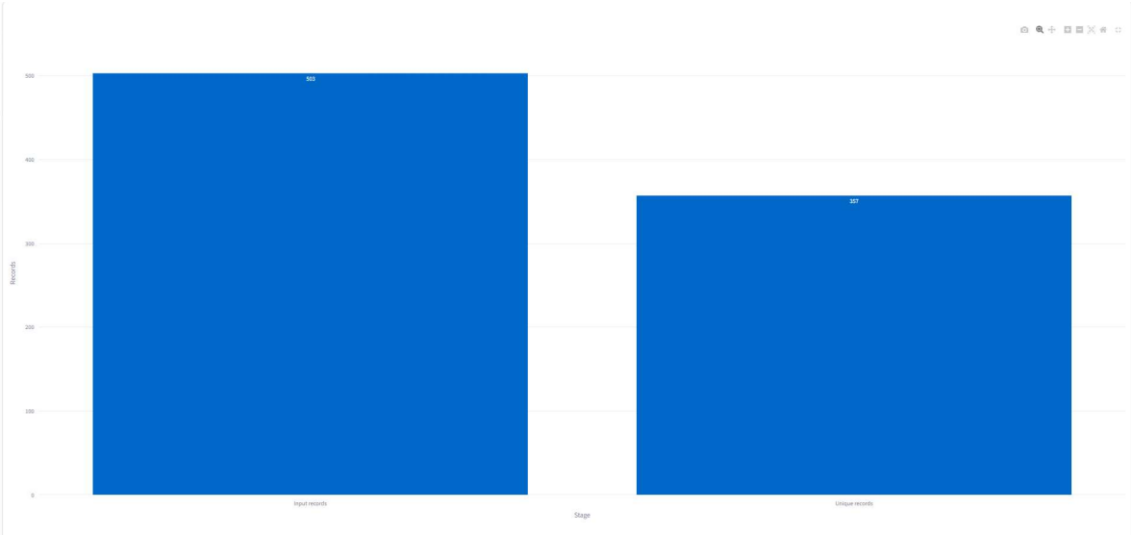


Fig. 5.5: Dataset reduction from 503 input records to 357 unique records

5.2 System Workflow / Data Flow Diagram

The data flow follows the pipeline from input to output. A CSV file or a synthetic dataset is validated and normalised, blocking produces candidate pairs, similarity features are computed for each pair, a classifier labels the pairs, union-find groups the linked records, and the results are exported and displayed.

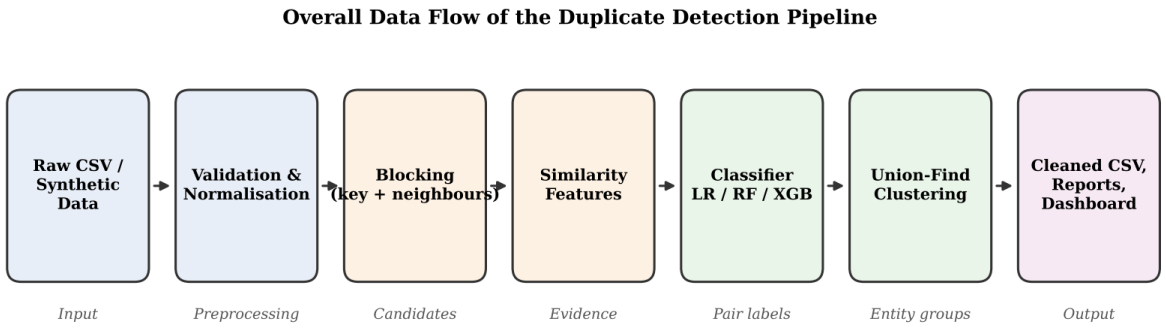


Fig. 5.6: Overall data flow of the duplicate detection pipeline

5.3 Dataset Design

To test duplicate detection under controlled conditions we used a generated customer dataset. The generator produces 503 rows: 350 original records and 153 injected duplicates, of which 65 are exact copies and 88 are near duplicates

with one or more altered attributes. It picks 122 underlying source records to duplicate, and 18 rows have at least one missing field. The result is a mix of easy exact-match cases and harder altered ones (Table 5.2).

Table 5.2: Generated dataset composition

Property	Count
Total rows	503
Original records	350
Injected duplicates	153
– exact copies	65
– near duplicates	88
Source records duplicated	122
Rows with a missing field	18

Each record has an identifier, name, email, phone, address and a duplicate_of field. The duplicate_of field gives the ground truth for building labels and for evaluation. It is never used as an input feature, so there is no target leakage.

5.4 Data Preprocessing Design

Missing text values are turned into empty strings for comparison, and the fields are normalised as shown in Table 5.3. For example, “999-673-4137” and “9996734137” become the same string after normalisation, so formatting alone can no longer hide a match.

Table 5.3: Normalisation rules by field

Field	Treatment	Effect
Name	Lowercase, collapse spaces, drop symbols	Ignores case/spacing
Email	Lowercase, collapse spaces, drop symbols	Ignores case
Address	Lowercase, collapse spaces, drop symbols	Ignores case/punctuation
Phone	Keep digits only	Ignores separators
Missing	Set to empty string; row flagged	Completeness check

Missing values need some care. Two empty fields must not count as an exact match, since having no information is not evidence that two records are the same person. The comparison should either record missingness separately or return a neutral score. A production version should also keep the original raw values next to the normalised ones, so that the cleaning stays auditable and reversible.

CHAPTER 6

Implementation

6.1 Module-wise Implementation

The project is divided into five major modules, each with a clear input and output.

Module 1: Data Ingestion and Preprocessing Module

Loads and validates the CSV and creates the normalised fields.

Input: Raw customer records. **Output:** Normalised records with missing-field flags.

Module 2: Candidate Generation Module

Builds blocking keys and adds neighbouring records after sorting by normalised name.

Input: Normalised records. **Output:** Candidate pairs.

Module 3: Similarity and Classification Module

Computes the pairwise feature vector and trains and evaluates Logistic Regression, Random Forest and XGBoost.

Input: Candidate pairs. **Output:** Feature table, predicted labels and metrics.

Module 4: Consolidation Module

Joins predicted links with union-find and keeps one representative per cluster.

Input: Predicted duplicate pairs. **Output:** Clusters and the cleaned unique dataset.

Module 5: Dashboard and Export Module

Shows metrics, charts, searchable pairs and CSV downloads.

Input: Pair table, metrics and clusters. **Output:** Charts, reports and downloadable files.

6.2 Data Ingestion and Validation

Input validation checks the required columns, rejects repeated identifiers, converts identifiers to integers and casts text fields to strings after filling missing values. For uploads without a usable duplicate_of field, the application applies the selected combined-score threshold and reports no supervised accuracy, precision or recall.

6.3 Candidate Generation (Blocking)

Comparing all 503 records with each other would need 503 × 502 / 2 = 126,253 pairs by Equation (1), and most of them are clearly unrelated. Our blocking strategy builds a key from the sorted first letters of the name tokens together with the first three digits of the normalised phone number, and records that fall in the same block become candidate pairs. To catch likely matches that straddle a strict block boundary, we also add neighbouring records after sorting by normalised name. This gave 605 candidate pairs, which Table 6.1 compares with the all-pairs baseline.

Table 6.1: Search-space reduction by blocking

Measure	Value
All possible pairs (n = 503)	126,253
Candidate pairs after blocking	605
Share of all pairs kept	≈ 0.48%
Pairs avoided	≈ 99.52%

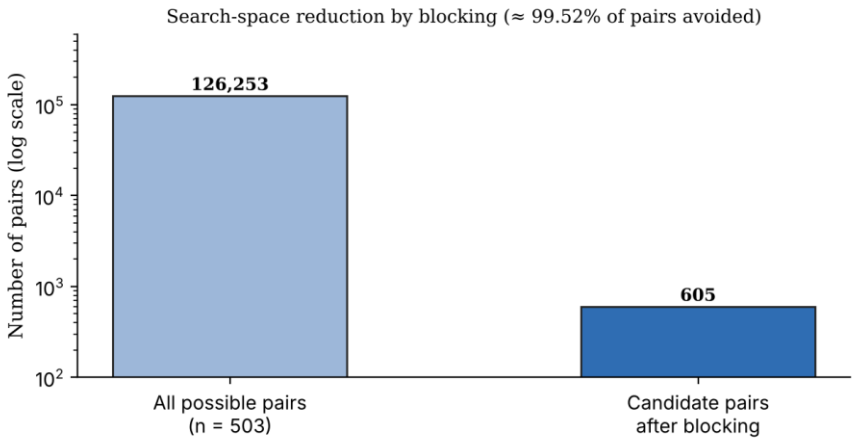


Fig. 6.1: Search-space reduction achieved by blocking

The last two rows are simple arithmetic on the reported counts. They measure efficiency only: a smaller candidate set does not tell us how many true duplicates survived blocking. That needs blocking recall (pair completeness), which the current outputs do not report. The exact key construction sits in a feature-engineering module that was not part of the code archive, so we report the observed 605 pairs and make no claim about the key beyond the description above.

6.4 Similarity Feature Engineering

For each candidate pair (A, B) the feature engine compares the matching attributes. Normalised Levenshtein similarity is

$$Slev(a, b) = 1 - d(a, b) / max(|a|, |b|) \quad (2)$$

where d(a, b) is the edit distance. Two empty strings are handled separately, and values close to 1 mean high similarity. Jaro–Winkler adds a prefix bonus to Jaro similarity and suits personal names, whose first characters are often stable, although a shared prefix can mislead if the measure is used alone. Jaccard similarity,

$$J(A, B) = |A \cap B| / |A \cup B| \quad (3)$$

compares token sets and so tolerates reordering; “Road 12, Central Area” and “Central Area, Road 12” contain the same tokens. TF–IDF maps text to weighted vectors, giving less weight to common words, and cosine similarity compares them:

$$cos(u, v) = (u \cdot v) / (\|u\| \|v\|) \quad (4)$$

We apply TF–IDF cosine over the combined textual fields. Exact-match indicators complement the approximate scores: phone_exact_match and email_exact_match equal one when the normalised values are identical. They are strong evidence but never proof, since contact details can be shared, recycled or missing. Table 6.2 lists the feature table we implemented.

Table 6.2: Pairwise feature catalogue

Feature	Measure	Captures
name_levenshtein	Edit distance	Small name typos
name_jaro_winkler	Jaro–Winkler	Stable name prefix
email_levenshtein	Edit distance	Email variants
address_levenshtein	Edit distance	Address typos
address_jaro_winkler	Jaro–Winkler	Address prefix
name_jaccard	Token overlap	Reordered name parts
address_jaccard	Token overlap	Reordered address parts
phone_exact_match	Binary	Same normalised phone
email_exact_match	Binary	Same normalised email
tfidf_cosine	Vector cosine	Weighted text overlap
combined_score	Weighted sum	Overall pair evidence

A weighted combined score can be written in general form as

$$C(A, B) = \Sigma k\, wk \cdot sk(A, B) \quad (5)$$

where the wk are non-negative weights and the sk are normalised comparison signals. The actual weights, scaling and threshold are specific to the implementation and are not visible in the dashboard, so they should be taken

from the source code when reproducing the experiment. We do not assign numerical weights here.

6.5 Classification Models

Every candidate pair becomes a numerical vector xij with a label saying whether the two records are the same entity. Logistic Regression, Random Forest and XGBoost are all evaluated on this same representation. Logistic Regression estimates

$$P(y = 1 \mid x) = 1 / (1 + exp(-(\beta 0 + \beta^{T} x))) \quad (6)$$

and a decision threshold turns the probability into a label. Table 6.3 compares the three classifiers.

Table 6.3: Classifier comparison

Model	Strength	Consideration
Logistic Regression	Interpretable baseline	Linear boundary
Random Forest	Non-linear interactions	Less transparent
XGBoost	Strong on tabular data	Sensitive to tuning

The original design uses a 70/30 train–test split and grid search with five-fold cross-validation. Tuning should use only the training partition so that the test set stays untouched for the final evaluation. Pair-level splitting needs care as well. If records of the same underlying entity end up in both partitions, leakage makes performance look better than it would on unseen entities, so a stronger evaluation would split by entity or source record.

6.6 Record-Level Consolidation

Predicted duplicate pairs are turned into groups with a union-find structure. Each record starts in its own set, and every predicted pair merges the sets of its two records. The resulting connected components are the candidate entity clusters. If A is linked to B and B to C, all three end up in one cluster even if the A–C

pair was never predicted. For each cluster the implementation keeps the record with the fewest missing fields as the representative and leaves the others out of the cleaned dataset.

This is simple and reproducible, but one false-positive edge can merge two distinct entities. Picking the most complete row also ignores recency, source authority and conflicting values. A production version should therefore add manual review, confidence thresholds, field-level survivorship rules and a reversible merge log. The overall steps, as seen in the dashboard source, are summarised in Table 6.4.

Table 6.4: Algorithm 1: classification and consolidation

Step	Action
1	Validate record_id, name, email, phone, address; reject duplicate or non-numeric IDs
2	Preprocess records; keep source attributes for display
3	Generate candidate pairs (blocking + adjacent names)
4	Compute the pairwise feature vector for each candidate
5	With enough labels: split, train, evaluate, pick best model, predict. Otherwise: apply threshold τ to the combined score
6	Build duplicate report from predicted matches and join original attributes
7	Create one disjoint set per record; union the endpoints of each predicted pair
8	Count missing fields; select one representative per component
9	Show metrics, charts, searchable pairs and CSV downloads

Most of the cost sits in the early stages. Exhaustive pairing is $O(n^2)$ in the number of pairs, while blocking aims for m far below $n(n-1)/2$. With bounded field lengths, feature construction is roughly $O(m \cdot L)$, where L is the average cost of one string comparison, and union-find does m unions over n records in near-linear amortised time. We did not run a formal runtime benchmark.

6.7 Prediction and Performance Analysis

After the features are computed, each model predicts a label for every candidate pair. The dashboard then evaluates the models using accuracy, precision, recall,

F1-score, a confusion matrix and a model comparison chart. For unlabelled uploads, only the combined-score threshold is applied and no supervised metrics are reported. The results are discussed in Chapter 8.

CHAPTER 7

Features

7.1 List of Main Features

The major features of the system are:

1. CSV upload or built-in synthetic dataset generation.
2. Input validation (required columns, repeated identifiers, type conversion).
3. Automatic text and phone normalisation with missing-field flagging.
4. Blocking with a neighbouring-name safety net.
5. Multi-field similarity features (Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine, exact match).
6. Side-by-side comparison of Logistic Regression, Random Forest and XGBoost.
7. Threshold control for unlabelled data.
8. Union-find clustering with one representative record per cluster.
9. Searchable pair table with scores and original attribute values.
10. Downloadable pair scores, duplicate report, cleaned dataset and metrics.
11. Asynchronous runs with progress updates in the Flask interface.

7.2 Working of Each Feature

Data Upload and Validation:

The user uploads a CSV or generates a synthetic dataset. The application checks the required columns, rejects repeated identifiers and prepares the text fields.

Normalisation:

Names, emails and addresses are lowercased with spaces collapsed and symbols dropped. Phones keep digits only. Missing values become empty strings and the row is flagged.

Blocking:

Records that share a blocking key become candidate pairs, and neighbours after sorting by name are added so that near-boundary matches are not lost.

Similarity Scoring:

Each candidate pair is described by character, token, vector and exact-match features, plus a combined score.

Classification:

Three classifiers are trained on the same features and compared. With no usable labels, the selected threshold is applied to the combined score instead.

Clustering:

Predicted pairs are merged into connected components, and the record with the fewest missing fields represents each cluster.

Inspection and Export:

The pair table can be searched, and the pair scores, duplicate report, cleaned dataset and metrics can be downloaded.

7.3 Interactive Dashboard

The dashboard keeps the model's decisions visible. The summary indicators show 503 input records, 605 candidate pairs, 172 flagged pairs and 357 unique records for the documented run. The score histogram shows non-duplicates at low and middle combined scores and flagged pairs at high scores, with a clear cluster near 1.0. The dashboard also warns that a flagged pair is only a candidate and recommends reviewing borderline matches, a safeguard worth keeping. See Fig. 5.1 to Fig. 5.5 in Chapter 5.

7.4 Output Artifacts

Keeping the pair scores, the duplicate report and the cleaned dataset as separate outputs helps traceability, because a reviewer can tell model evaluation apart from the final consolidation result. Table 7.1 lists the files produced.

Table 7.1: Output artifacts

File	Content
candidate_pair_scores.csv	All candidate pairs with features and labels (605 rows, 15 columns)
duplicate_records_report.csv	Predicted duplicate pairs with readable values
cleaned_unique_dataset.csv	One representative per cluster
model_performance_metrics.json	Classifier metrics
Charts	Model comparison, confusion matrix, score distribution, dataset size

CHAPTER 8

Testing

8.1 Functional Testing

Functional testing checks whether the main features of the system behave as expected. The following operations are covered:

1. Uploading a CSV and generating a synthetic dataset.
2. Validating required columns and identifiers.
3. Normalising names, emails, addresses and phones.
4. Generating candidate pairs through blocking.
5. Computing the pairwise feature table.
6. Training and comparing the three classifiers.
7. Flagging duplicate pairs and building clusters.
8. Displaying charts and the searchable pair table, and downloading outputs.

8.2 Integration Testing

Integration testing checks whether the different modules work together. The following sequence is verified on the documented run:

1. The 503-record dataset is validated and normalised.
2. Blocking produces 605 candidate pairs from 126,253 possible pairs.
3. The feature table (605 rows, 15 columns) is passed to the three classifiers.
4. The best model flags 172 pairs as duplicates.
5. Union-find groups the linked records and leaves 357 representatives.
6. Metrics, charts and the pair table are displayed in the dashboard.

8.3 Tools Used for Testing

- Python for executing the pipeline.

- scikit-learn for model evaluation (accuracy, precision, recall, F1-score, confusion matrix).
- pandas and NumPy for checking counts and tables.
- Streamlit and Plotly for inspecting results visually.
- matplotlib and seaborn for the supporting charts.

Postman testing is not applicable because the system does not expose a separate REST API for external clients.

8.4 Test Cases and Results

Table 8.1 lists the test cases. The status column separates results that are visible in the documented run from behaviour that is described by the design but still needs to be re-verified with a fresh run.

Table 8.1: Test cases and results

ID	Test Case	Expected Result	Status
TC-01	Upload data with repeated or non-numeric IDs	Input is rejected with a validation message	Designed behaviour
TC-02	Normalise “999-673-4137” and “9996734137”	Both become the same string	Documented example
TC-03	Run blocking on 503 records	126,253 possible pairs reduced to 605 candidates	Observed
TC-04	Build the pair feature table	605 rows and 15 columns saved	Observed
TC-05	Compare the three classifiers	Metrics reported for all three models	Observed
TC-06	Flag duplicates and cluster	172 flagged pairs, 357 representatives	Observed
TC-07	Open the dashboard	Indicators, charts and pair table displayed	Observed
TC-08	Upload data without duplicate_of	Threshold applied, no supervised metrics	Designed behaviour
TC-09	Download outputs	Pair scores, report, cleaned CSV and metrics available	Designed behaviour

8.5 Model Performance Evaluation

Record reduction and the flagged-pair rate measure different things, as Table 8.2 shows. The first is a reduction in record count, and the second is the share of candidate pairs classified as duplicates. Note that 172 links reduced the row count by only 146, because links overlap inside clusters: a cluster of three records can produce several links while removing just two rows. The figure of 146 should not be read as the true number of duplicate entities unless the ground-truth clusters are checked independently.

Table 8.2: Reduction and flagging statistics

Measure	Calculation	Result
Records removed	503 − 357	146
Record reduction	146 / 503 × 100	29.02%
Flagged-pair rate	172 / 605 × 100	28.43%

The dashboard reports identical rounded metrics for all three models (Table 8.3) and names Logistic Regression as the best one. Because the values are rounded to three decimals, the screenshot cannot show whether the unrounded scores are identical or how the tie was broken. We therefore say the models performed similarly instead of calling one of them superior.

Table 8.3: Displayed performance of the classifiers

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.995	1.000	0.981	0.990
Random Forest	0.995	1.000	0.981	0.990
XGBoost	0.995	1.000	0.981	0.990

The confusion matrix for Logistic Regression (Table 8.4) covers 182 candidate pairs. Accuracy is (130 + 51)/182 = 0.9945, precision is 51/(51 + 0) = 1.0000, recall is 51/(51 + 1) = 0.9808, and the harmonic mean of precision and recall is about 0.9903. All of these round to the dashboard values. How the 182 pairs were selected cannot be established from the available evidence.

Table 8.4: Confusion matrix (Logistic Regression)

	Pred. non-dup.	Pred. dup.
Actual non-duplicate	130 (TN)	0 (FP)
Actual duplicate	1 (FN)	51 (TP)

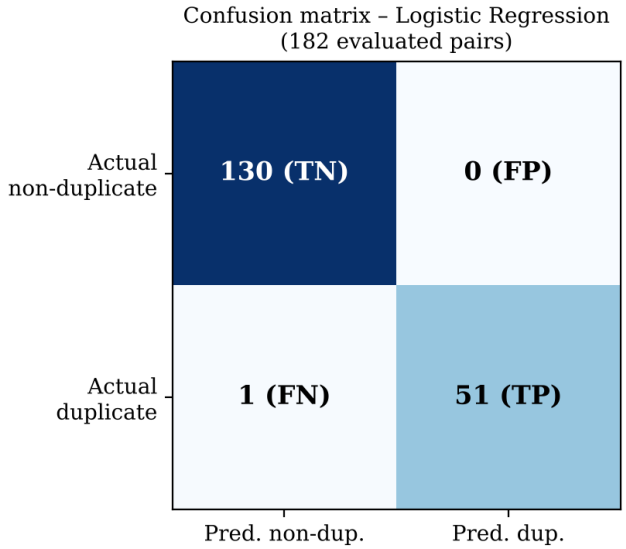


Fig. 8.1: Confusion matrix for Logistic Regression on 182 evaluated pairs

No false positive appears in the displayed sample, and one true duplicate was missed. The two kinds of error do not cost the same (Table 8.5), so the threshold should be chosen to suit the intended application.

Table 8.5: Cost of each error type

Error	Consequence	Mitigation
False positive	Distinct people merged	Higher threshold, review
False negative	Duplicate row survives	Lower threshold, rerun

The result is promising for the generated sample, but it does not show that the system generalises to real customer datasets. That would need independent labels, leakage-safe splitting, repeated validation and external data.

8.6 Threats to Validity

The main threats to the validity of these results are summarised in Table 8.6.

Table 8.6: Threats to validity

Threat	Why it matters	Remedy
Synthetic-data bias	Corruptions may be simpler than real ones	Real labelled data
Entity leakage	Same entity in train and test inflates scores	Entity-level split
Candidate selection	Classifier only sees pairs that survive blocking	Measure blocking recall
Rounded metrics	Ties cannot be resolved	Report unrounded scores
Unknown split protocol	Seed and split not shown	Log full configuration
Missing source modules	Weights and keys unverified	Publish full source
Cluster propagation	One bad link merges entities	Review, reversible merges

Blocking needs its own evaluation. Pair completeness (blocking recall) is the share of all known true duplicate pairs that appear in the candidate set, and pair quality is the share of candidate pairs that are true matches. A strict rule gives a small, clean candidate set but may miss matches, while a permissive rule keeps more matches but puts more load on the classifier. Future work should report both measures.

CHAPTER 9

Deployment

9.1 Steps to Run the Application

The application can be executed locally using the following steps:

1. Install Python 3.x on the computer.
2. Open the project folder in Visual Studio Code.
3. Create a Python virtual environment.
4. Activate the virtual environment.
5. Install the required Python libraries.
6. Start the Flask application, or start the Streamlit dashboard.
7. Open the local URL shown in the terminal in a web browser.

Command to start the Streamlit dashboard (replace the file name with your dashboard script):

```
python -m streamlit run <dashboard_script>.py
```

9.2 Python Environment Configuration

A Python virtual environment keeps the project libraries separate from the rest of the system. The environment needs the packages listed in Table 2.1: pandas, NumPy, scikit-learn, XGBoost, Python-Levenshtein, jellyfish, matplotlib, seaborn, Flask, Streamlit and Plotly.

9.3 Local Application Execution

The system is built to run locally for experiments. The interface and the processing logic run in the same Python environment, so separate front-end and back-end hosting is not required for the current implementation.

9.4 Production Readiness

Table 9.1 summarises what a production version would still need. Customer records are personal data, so access should be restricted, data protected in transit and at rest, full contact details kept out of logs, and a retention policy defined. Duplicate detection should never delete source records without review, especially where billing, service or legal records are involved.

Table 9.1: Production readiness checklist

Area	Current state	Recommended addition
Security	Local experiment	Authentication, roles, TLS
Auditability	CSV exports	Log model, threshold, reviewer
Review	Searchable table	Approve / reject / defer queue
Merges	Keep fewest-missing row	Field-level survivorship, undo
Scale	503 records	Batch, DB-backed, incremental
Monitoring	Score histogram	Drift and flag-rate tracking
Reproducibility	Partial	JSON manifest, seeds, versions

CHAPTER 10

Challenges and Limitations

10.1 Issues Faced During Development

Messy Text Variations:

The same person can appear with typos, different capitalisation, abbreviations, reordered address words or reformatted phone numbers, so no single comparison is enough.

Search-Space Growth:

Comparing all 503 records needs 126,253 pairs, and the number grows quadratically with the data size.

Missing Values:

Two empty fields must not be treated as a match, because having no information is not evidence of identity.

Pairs versus Rows:

One record can take part in several duplicate links, so the number of flagged pairs (172) is not the number of rows removed (146).

Identical Metrics:

All three classifiers reported the same rounded scores, which made it impossible to rank them.

Limited Verification:

Some source modules were not part of the archive, so blocking keys, feature weights, split settings and hyperparameters could not be independently verified.

10.2 Solutions Applied

1. Text and phone normalisation was used to remove formatting differences.
2. Blocking with a neighbouring-name safety net was used to cut the number of comparisons.
3. Several similarity measures were combined so that different kinds of error are covered.
4. Missing values were turned into empty strings and the row was flagged for a completeness check.
5. Union-find clustering was added, and pair counts and row counts are reported separately.
6. The three models are described as performing similarly instead of declaring a winner.
7. The duplicate_of field was used only for labels and never as an input feature, to avoid target leakage.
8. Threats to validity were written down together with a remedy for each.

10.3 Current Limitations

Several limitations follow from the evidence we have:

1. The dataset is generated, and synthetic duplicates are made by copying a row and applying a limited set of transformations, so a model may simply learn those transformations.
2. The exact combined-score weights and threshold are not visible.
3. The unrounded per-model metrics and the random seed are not shown.
4. A pair-level random split may leak entities between training and test sets.
5. Blocking recall (pair completeness) has not been measured.
6. A key built from name initials and phone prefixes may fail when both fields are corrupted.
7. Cluster consolidation can merge two entities because of one false-positive link, and picking the most complete row ignores recency and source authority.
8. The system is built for local experiments and is not ready for production use.

The reported figures should therefore be read as results on the current generated evaluation sample and not as a general expected accuracy.

CHAPTER 11

Future Enhancements

11.1 Planned Features

The most important next step is a stronger experiment and not another model: leakage-safe splitting, external data, blocking-recall measurement and a reviewable merge process. Table 11.1 lays out the planned directions.

Table 11.1: Future work roadmap

Direction	Planned work
Evaluation	Real datasets, human labels, entity-level splits, repeated stratified validation, PR curves, confidence intervals
Blocking	Measure pair completeness, improve candidates
Modelling	Learn feature weights, calibrate thresholds, try transformer matchers
Human review	Active learning, approve / reject workflow
Consolidation	Survivorship rules, reversible merges, audit logs
Scalability	Incremental matching against existing entities

11.2 Learned Weights and Transformer Matching

The combined score currently uses fixed weights. Learning these weights from labelled data and calibrating the decision threshold would make the score more reliable. Transformer-based matchers [12], [13] could also be added as an extra model on top of the same candidate pairs, since the classical features were chosen on purpose to leave room for them.

11.3 Human Review and Scalability

Uncertain pairs should go to a review queue where a person can approve, reject or defer them, and active learning can use those decisions to improve the model. Error analysis by corruption type, such as name typo, phone formatting, missing email or address variation, would show where the features need work. For larger

data, the pipeline should move to batch, database-backed and incremental matching against existing entities.

CHAPTER 12

Conclusion

12.1 Summary of the Project

This project presented a similarity-based duplicate record detection system for structured customer data. It combines normalisation, blocking, multi-field similarity features, supervised classification, duplicate clustering and an interactive dashboard, using Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine and exact phone and email signals.

In the documented run, 503 input records produced 605 candidate pairs, 172 flagged pairs and 357 representative records, a reduction of 146 rows or about 29.0%. All three classifiers showed 0.995 accuracy, 1.000 precision, 0.981 recall and 0.990 F1, with 130 true negatives, no false positives, one false negative and 51 true positives. These are strong results on the displayed sample, but they should be read in light of the synthetic data and the limited validation evidence.

12.2 Achievements

1. Built a modular end-to-end deduplication pipeline from raw CSV to cleaned dataset.
2. Reduced 126,253 possible pairs to 605 candidates with blocking.
3. Designed an 11-feature, multi-field representation of record pairs.
4. Compared Logistic Regression, Random Forest and XGBoost on one shared feature set.
5. Converted pairwise predictions into clusters and a cleaned dataset with 357 representative records.
6. Reported pair counts and row counts separately to avoid confusion.
7. Built a dashboard with charts, a searchable pair table and downloadable outputs.
8. Documented the threats to validity and a roadmap for stronger evaluation.

12.3 Skills Learned During Development

Through this project, we gained practical experience in:

1. Python programming.
2. Text preprocessing and normalisation.
3. String similarity measures and TF–IDF.
4. Blocking and candidate generation.
5. Machine learning classification and model comparison.
6. Graph clustering with union-find.
7. Flask and Streamlit application development.
8. Data visualization with matplotlib, seaborn and Plotly.
9. Critical evaluation and honest reporting of results.
10. Team collaboration and technical documentation.

CHAPTER 13

References

1. I. P. Fellegi and A. B. Sunter, “A theory for record linkage,” J. Amer. Statist. Assoc., vol. 64, no. 328, pp. 1183–1210, 1969.
2. V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Phys. Doklady, vol. 10, pp. 707–710, 1966.
3. M. A. Jaro, “Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida,” J. Amer. Statist. Assoc., vol. 84, no. 406, pp. 414–420, 1989.
4. W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage,” in Proc. Sect. Survey Res. Methods, Amer. Statist. Assoc., 1990, pp. 354–359.
5. G. Salton and M. J. McGill, Introduction to Modern Information Retrieval. New York, NY, USA: McGraw-Hill, 1983.
6. P. Christen, Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Berlin, Germany: Springer, 2012.
7. P. Christen, “A survey of indexing techniques for scalable record linkage and deduplication,” IEEE Trans. Knowl. Data Eng., vol. 24, no. 9, pp. 1537–1555, 2012.
8. G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, “Blocking and filtering techniques for entity resolution: A survey,” ACM Comput. Surv., vol. 53, no. 2, 2020.
9. M. Bilenko and R. J. Mooney, “Adaptive duplicate detection using learnable string similarity measures,” in Proc. ACM SIGKDD, 2003, pp. 39–48.
10. L. Breiman, “Random forests,” Mach. Learn., vol. 45, pp. 5–32, 2001.
11. T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. ACM SIGKDD, 2016, pp. 785–794.
12. S. Mudgal et al., “Deep learning for entity matching: A design space exploration,” in Proc. ACM SIGMOD, 2018, pp. 19–34.

13. Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, “Deep entity matching with pre-trained language models,” arXiv:2004.00584, 2020.
14. W. W. Cohen, P. Ravikumar, and S. E. Fienberg, “A comparison of string distance metrics for name-matching tasks,” in Proc. IJCAI, 2003, pp. 73–78.
15. A. Arasu, V. Ganti, and R. Kaushik, “Efficient exact set-similarity joins,” in Proc. VLDB, 2006.

CHAPTER 14

Appendices

14.1 Application Screenshots

The following screenshots demonstrate the working of the application. They are shown in Chapter 5:

1. Dashboard overview with summary indicators.
2. Model evaluation comparison for the three classifiers.
3. Confusion matrix and detected duplicate-pair table.
4. Distribution of combined similarity scores.
5. Dataset reduction from 503 to 357 records.

14.2 Sample Code Snippets

The following simplified snippets illustrate the ideas used in the pipeline. They were written for this report to explain the logic and are not the project's source code.

```
import re
from collections import defaultdict

def normalise_phone(value):
    return re.sub(r"\D", "", value or "")          # keep digits only

def lev_similarity(a, b):
    # Equation (2): 1 - d(a, b) / max(|a|, |b|)
    if not a and not b:
        return 0.0                                 # two empty fields are
not a match
    d = edit_distance(a, b)                        # e.g. from python-
Levenshtein
    return 1 - d / max(len(a), len(b))

def jaccard(a, b):
    A, B = set(a.split()), set(b.split())
    return len(A & B) / len(A | B) if A | B else 0.0


# Union-find: merge predicted duplicate pairs into clusters
parent = {}
def find(x):
    parent.setdefault(x, x)
    while parent[x] != x:
```

```
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def union(a, b):
    parent[find(a)] = find(b)

for a, b in predicted_duplicate_pairs:
    union(a, b)

clusters = defaultdict(list)
for rid in all_record_ids:
    clusters[find(rid)].append(rid)
# keep the record with the fewest missing fields in each cluster
```

14.3 Installation and Setup Instructions

Step 1: Install Python 3.x.

Step 2: Open the project folder in Visual Studio Code.

Step 3: Create and activate the virtual environment.

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Step 4: Install the required libraries.

```
pip install pandas numpy scikit-learn xgboost python-Levenshtein
jellyfish matplotlib seaborn flask streamlit plotly
```

Step 5: Run the application (replace the file names with your own scripts).

```
python <flask_app>.py
python -m streamlit run <dashboard_script>.py
```

Step 6: Open the local URL displayed in the terminal.

14.4 User Manual

The application can be used through the following steps:

1. Open the application in a browser.
2. Generate a synthetic dataset or upload a CSV with record_id, name, email, phone and address columns.
3. Wait for the run to finish; progress is shown on screen.
4. For unlabelled data, adjust the similarity-threshold slider.
5. Review the summary indicators and the score histogram.

6. Compare the three classifiers and check the confusion matrix.
7. Search the detected duplicate-pair table and review borderline pairs.
8. Download the pair scores, duplicate report, cleaned dataset and metrics.

14.5 Team Details

Table 14.1: Project team

Name	Roll Number	Role
Allipuram Akhil	2420030540	Student, B.Tech CSE
D. Surya	2420030440	Student, B.Tech CSE
P. Ruthvik Reddy	2420030546	Student, B.Tech CSE
Ms. Bonagani Prathusha	—	Project Guide, Assistant Professor, CSE

All team members contributed to project design, implementation, integration, testing, debugging and documentation under the guidance of Ms. Bonagani Prathusha.